

เกมปูกระเบื้อง

บัคซินและชาร์อสเล่นเกมบนตารางขนาด $2N \times 2M$ ช่อง แถวต่างๆ จะมีหมายเลขตั้งแต่ 0 ถึง $2N - 1$ จากบนลงล่าง และคอลัมน์ต่างๆ จะมีหมายเลขตั้งแต่ 0 ถึง $2M - 1$ จากซ้ายไปขวา เราจะใช้สัญลักษณ์ (i, j) แทนช่องที่อยู่ในแถว i และคอลัมน์ j สำหรับ $0 \leq i < 2N$ และ $0 \leq j < 2M$

บัคซินมอบบล็อกจำนวน $N \cdot M$ ชิ้น ให้กับชาร์อสทีละชิ้น แต่ละชิ้นเป็นสี่เหลี่ยมจัตุรัสขนาด 2×2 ที่ประกอบด้วยกระเบื้องขนาด 1×1 จำนวน 4 แผ่น โดยบัคซินได้ระบายสีกระเบื้องแต่ละแผ่นในบล็อกด้วยสีดำหรือสีขาว โดยที่รับประกันว่า มีกระเบื้องอย่างน้อยหนึ่งแผ่นเป็นสีขาว

ชาร์อสต้องวางบล็อกแต่ละชิ้นลงบนตารางทันทีหลังจากได้รับ โดยที่เธอไม่รู้ว่บล็อกต่อไปจะเป็นอะไร เธอไม่สามารถหมุนบล็อกได้ และบล็อกแต่ละชิ้นต้องวางอยู่ภายในตารางเต็มทั้งชิ้น โดยคลุมช่อง 4 ช่องพอดี นอกจากนี้ ช่องบนซ้ายของแต่ละบล็อกจะต้องอยู่ในช่องที่มีพิกัดทั้งแถวและคอลัมน์เป็นเลขคู่ แต่ละช่องในตารางสามารถถูกคลุมโดยบล็อกได้ไม่เกิน 1 ชิ้น

บัคซินจะชนะเกมเมื่อ หลังจากวางบล็อกชิ้นใดๆ แล้ว เกิดช่องสี่เหลี่ยมจัตุรัสขนาด 2×2 ที่ถูกคลุมด้วยกระเบื้องสีดำ 4 แผ่น กล่าวคือ ถ้าช่อง (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ ถูกคลุมด้วยกระเบื้องสีดำทั้งหมด โดยที่ $0 \leq a < 2N - 1$ และ $0 \leq b < 2M - 1$ แล้วบัคซินจะชนะเกม ดัชนี a และ b ไม่จำเป็นต้องเป็นเลขคู่ก็ได้

ชาร์อสจะชนะเกมเมื่อ เธอวางบล็อกครบทั้ง $N \cdot M$ ชิ้นโดยที่บัคซินไม่ชนะ สังเกตว่าการวางบล็อก $N \cdot M$ ชิ้นจะคลุมทั่วทั้งตารางพอดี

งานของคุณคือการวางกลยุทธ์ให้ชาร์อสชนะเกม เราสามารถพิสูจน์ได้ว่าชาร์อสมีวิธีวางบล็อกให้ชนะเกมได้เสมอ ไม่ว่าบล็อกที่เธอได้รับจะมีสีอะไรบ้างก็ตาม

รายละเอียดการเขียนโปรแกรม

คุณต้องเขียนฟังก์ชัน 2 ฟังก์ชันต่อไปนี้:

```
void init(int N, int M)
```

- N : ครึ่งหนึ่งของจำนวนแถวในตาราง
- M : ครึ่งหนึ่งของจำนวนคอลัมน์ในตาราง
- ฟังก์ชันนี้จะถูกเรียกใช้เพียงครั้งเดียวต่อข้อมูลทดสอบ ในช่วงแรกสุดของการทำงานของโปรแกรม

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : สีของกระเบื้องช่องบนซ้าย บนขวา ล่างซ้าย และล่างขวาในบล็อกปัจจุบัน ตามลำดับ ดังที่แสดงในรูปด้านล่าง แต่ละค่าเป็น 0 (สีขาว) หรือ 1 (สีดำ)
- ฟังก์ชันนี้จะถูกเรียกใช้ $N \cdot M$ ครั้งพอดีต่อข้อมูลทดสอบ หลังจากเรียกใช้ฟังก์ชัน `init`



ฟังก์ชันนี้จะต้องส่งออกคู่อันดับของจำนวนเต็ม (i, j) โดยที่ i คือพิกัดแถว และ j คือพิกัดคอลัมน์ของเซลล์ที่จะวางกระเบื้องช่องบนซ้ายของบล็อกนี้ ทั้ง i และ j ต้องเป็นจำนวนคู่ และพื้นที่ขนาด 2×2 ที่บล็อกนั้นคลุมจะต้องไม่ทับซ้อนกับบล็อกอื่นที่วางไว้ก่อนหน้านี้

ถ้า `receive_block` ส่งออกคู่อันดับที่ไม่ตรงตามเงื่อนไขเหล่านี้ หรือหากหลังจากวางบล็อกแล้ว เกิดช่องสี่เหลี่ยมจัตุรัสขนาด 2×2 ที่ถูกคลุมด้วยกระเบื้องสีดำทุกช่อง เกรดเดอร์จะยุติการทำงานทันที และผลลัพธ์จะเป็น `Output isn't correct`

พฤติกรรมของเกรดเดอร์จะ**ไม่ปรับเปลี่ยน (ไม่ adaptive)** นั้นหมายความว่าลำดับของบล็อกที่บัคซินมอบให้หารอสได้ถูกกำหนดไว้ตายตัวแล้วตั้งแต่ก่อนเรียกฟังก์ชัน `init`

เงื่อนไข

สำหรับแต่ละบล็อก ให้ S เป็นจำนวนของกระเบื้องสีดำในบรรดากระเบื้องทั้ง 4 แผ่นในบล็อกนั้น นั่นคือ $S = TL + TR + BL + BR$

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ สำหรับทุกบล็อก

ปัญหาย่อย

ปัญหาย่อย	คะแนน	เงื่อนไขเพิ่มเติม
1	6	$S = 1$ สำหรับทุกบล็อก และ $N = 2$
2	16	$S = 3$ สำหรับทุกบล็อก และ $N = M$ และ N เป็นเลขคู่ และหน้าตาของบล็อกทั้ง 4 แบบที่เป็นไปได้จะปรากฏแบบละ $\frac{N^2}{4}$ บล็อกพอดี
3	10	$S = 1$ สำหรับทุกบล็อก
4	29	$S \leq 2$ สำหรับทุกบล็อก
5	39	ไม่มีเงื่อนไขเพิ่มเติม

ตัวอย่าง

พิจารณาเกมที่ $N = 1$ และ $M = 2$ ตารางจะมี 2 แถวและ 4 คอลัมน์ เริ่มมาเกรตเตอร์จะเรียกฟังก์ชัน:

```
init(1, 2)
```

ในตอนเริ่มต้น ช่องทุกช่องว่างเปล่า ตารางจะมีหน้าตาแบบนี้:

	0	1	2	3
0				
1				

มีบล็อกจำนวน $N \cdot M = 2$ ชิ้นที่ต้องวาง สมมติว่าบล็อกชิ้นมอบบล็อกที่มีกระเบื้องสีดำ 3 แผ่น และกระเบื้องสีขาว 1 แผ่น อยู่ที่มุมบนขวา

```
receive_block(1, 0, 1, 1)
```

ชาวอสังวบบล็อกนี้ที่ด้านซ้ายของตาราง โดยส่งออก (0,0)

ตอนนี้ตารางจะมีหน้าตาแบบนี้:

	0	1	2	3
0				
1				

จากนั้น บล็อกชิ้นมอบบล็อกอีกชิ้นที่มีกระเบื้องสีดำ 3 แผ่น และกระเบื้องสีขาว 1 แผ่น อยู่ที่มุมบนขวา

```
receive_block(0, 1, 1, 1)
```

ช่องเดียวที่เหลืออยู่ซึ่งมีแถวและคอลัมน์เป็นเลขคู่และสามารถใช้เป็นมุมบนซ้ายของบล็อกขนาด 2×2 ได้คือช่อง (0,2) ชาวอสังวบบล็อก (0,2) ตารางจึงมีหน้าตาแบบนี้:

	0	1	2	3
0				
1				

ไม่มีสี่เหลี่ยมจัตุรัสขนาด 2×2 ที่ถูกรวมด้วยกระเบื้องสีดำทุกช่อง ดังนั้น ชาโรสจึงวางบล็อกได้ครบทุกชิ้นโดยที่บัคซินไม่ชนะ ดังนั้น ชาโรสจึงชนะเกม

เกร็ดเดอร์ตัวอย่าง

รูปแบบข้อมูลนำเข้า:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

รูปแบบข้อมูลส่งออก:

```
R[0] C[0]
R[1] C[1]
...
R[NM-1] C[NM-1]
```

ในที่นี้ $R[k]$ และ $C[k]$ คือคู่อันดับของจำนวนเต็มที่ส่งออกในการเรียก `receive_block` ครั้งที่ k